LAB 5 Uninformed Search – II

5.1 Uniform Cost Search:

Instead of expanding the shallowest node as in BFS, **uniform-cost search** expands the node n with the *lowest path cost* g(n). This is done by storing the frontier as a priority queue ordered by g. The algorithm is shown below. At each stage, the path that has the lowest cost so far is extended. In this way, the path that is generated is likely to be the path with the lowest overall cost.

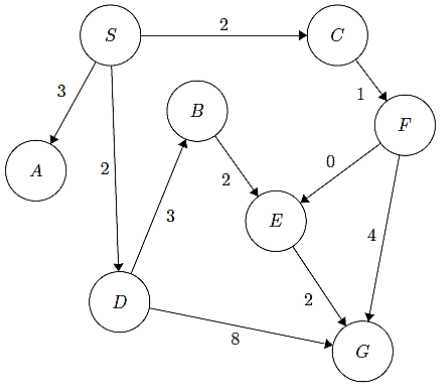
```
function UNIFORM-COST-SEARCH(problem) returns a solution, or failure
  node ← a node with STATE = problem.INITIAL-STATE, PATH-COST = 0
  frontier ← a priority queue ordered by PATH-COST, with node as the only element
  explored ← an empty set
  loop do
    if EMPTY?(frontier) then return failure
    node ← POP(frontier)  /* chooses the lowest-cost node in frontier */
    if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
    add node.STATE to explored
    for each action in problem.ACTIONS(node.STATE) do
      child ← CHILD-NODE(problem, node, action)
      if child.STATE is not in explored or frontier then
        frontier ← INSERT(child, frontier)
      else if child.STATE is in frontier with higher PATH-COST then
        replace that frontier node with child
```

Figure 5-1. Uniform-cost search on a graph use of a priority queue and the addition of an extra check in case a shorter path to a frontier state is discovered.

5.2 Lab Tasks

Exercise 5.1.

Modify the code of Breadth First Search implemented in LAB 04 to Uniform Cost Search. Display the order in which nodes are added to the explored set, with **start state S** and **goal state G**. Path costs are mentioned on the arcs. Break ties in alphabetical order. What is the total cost of path found using UCS?



Exercise 5.2.

In the following exercises a weighted graph is stored in Python as a dictionary of dictionaries: each key is a node and its inner dictionary maps every node that it points to onto the cost of that edge. All edges are one-way, so if A points to B you can go from A to B, but not from B to A. In the figures, the number on an edge is its cost. Graph 1 (Figure 5-2) is stored as follows.

```
graph1 = {'S': {'A': 1, 'B': 5, 'D': 3}, 'A': {'C': 2},
          'B': {'G': 10}, 'C': {'G': 3}, 'D': {'G': 9}, 'G': {}}
```

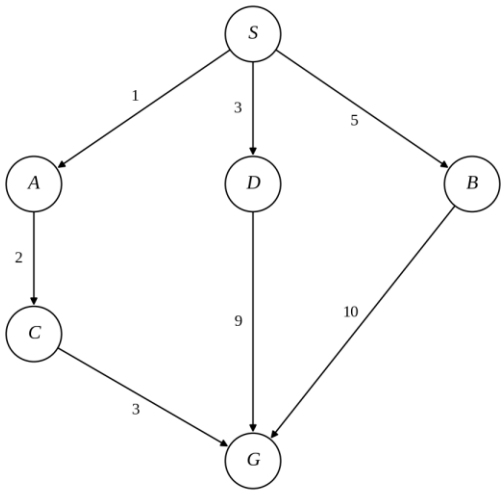


Figure 5-2 Graph 1 (weighted directed graph)

Implement `ucs(start, goal)` using Uniform Cost Search. Use a priority queue ordered by the path cost g(n) (for example `heapq` or a list that is sorted by g(n)), break ties alphabetically, and test for the goal when a node is removed from the frontier. The function reads the global variable `graph` and returns a tuple `(cost, path)` holding the total cost and the path from the start to the goal, or `(float('inf'), None)` if the goal cannot be reached. It must also print the order in which the nodes are added to the explored set.

Set `graph = graph1` and run `ucs('S', 'G')`.

Exercise 5.3.

Graph 2 (Figure 5-3) has a node that can be reached by two different paths (A can be reached directly from S or through B) and a cycle (A points to C, C points to B and B points back to A). It is stored as follows.

```
graph2 = {'S': {'A': 4, 'B': 1}, 'A': {'C': 3, 'G': 8},
          'B': {'A': 2}, 'C': {'B': 1, 'G': 4}, 'G': {}}
```

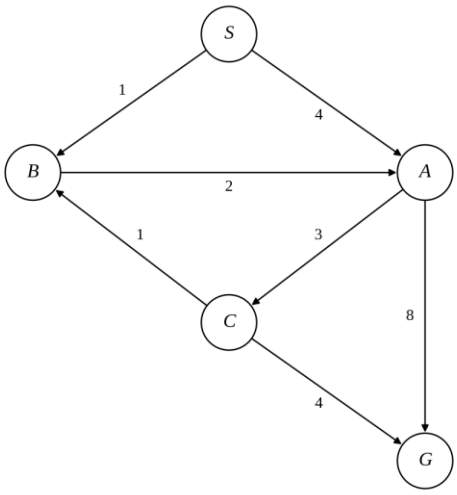


Figure 5-3 Graph 2 (weighted directed graph with a cycle)

Set `graph = graph2` and run `ucs('S', 'G')`. Make sure that a node which is reached again by a cheaper path uses the cheaper cost (the old frontier entry is replaced or ignored) and that a node which is already in the explored set is never expanded again. Print the order in which the nodes are added to the explored set, the path and its cost.

Exercise 5.4.

Graph 3 (Figure 5-4) has seven nodes and several paths to G. It is stored as follows.

```
graph3 = {'S': {'A': 2, 'B': 3}, 'A': {'D': 4},
          'B': {'C': 1, 'D': 2}, 'C': {'E': 5},
          'D': {'E': 2, 'G': 6}, 'E': {'G': 1}, 'G': {}}
```

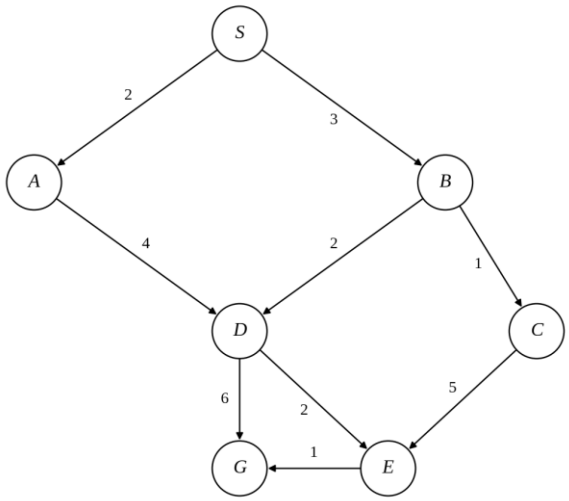


Figure 5-4 Graph 3 (weighted directed graph)

Set `graph = graph3` and run `ucs('S', 'G')`. Print the order in which the nodes are added to the explored set, the path and its cost. Then run `ucs('G', 'S')` and explain the result.

Exercise 5.5.

Write a function `ucs_all(start)` that does not stop at a goal. It keeps expanding the cheapest node until the frontier is empty and returns a dictionary that maps every reachable node onto its lowest path cost from `start`.

Run `ucs_all('S')` on `graph3` and print the dictionary. Which node is the farthest from S?